

자기소개서

AI Solution Engineer - Cloud

이기찬 | 010-6666-2824 | gichanlee@icloud.com

포트폴리오: <https://bit-habit.com/?focus=solutions&title=AI%20Solution%20Engineer> | GitHub: github.com/bookseal

1. "한 번 만든 구현이 템플릿이 된다" — 제가 일해온 방식입니다

채용공고에서 가장 오래 붙잡고 읽은 문장이 있습니다.

"한 번 만든 구현이 템플릿이 되고, 그 템플릿이 다음 달의 배포 속도를 결정합니다."

저는 이 문장을 직무 설명이 아니라 지난 2 년간 일해온 방식의 요약으로 읽었습니다.

k3s 클러스터의 첫 서비스는 매니페스트를 손으로 썼고, 두 번째에서 복사를 멈추고 ArgoCD 기반 GitOps 로 선언형 파이프라인을 표준화했습니다. 지금은 서비스가 14 개인데, **14 번째를 올리는 비용이 2 번째보다 쌉니다.**

인그레스·TLS 는 Traefik + cert-manager 가 자동으로 붙고, 배포는 PR 병합이 끝입니다.

같은 일을 현직에서도 하고 있습니다. 사내 피드백 대시보드는 하드코딩 대신 검증된 오픈소스로 세팅했고, 매년 갱신되는 정부 대가 산정 가이드는 크롤링·동기화 파이프라인으로 사람 손을 뺐습니다. **반복을 더 빨리 하는 대신 반복 자체를 없애는 쪽을 택합니다.**

2. LLM 을 실제로 서빙해 봤습니다 — 온디맨드로, 비용 0 으로

이 포지션의 본질이 **"AI 모델이 고객의 프로덕션 환경에서 안정적으로 서빙되기까지의 전체 인프라 라이프사이클"** 이라고 이해했습니다. 컨테이너 오케스트레이션, 리소스 최적화, CI/CD, 모니터링, 오토스케일링.

저는 이 루프를 **llm-app-lab**(llm-app-lab.bit-habit.com)에서 처음부터 끝까지 직접 만들었습니다.

정적 문서 페이지에는 서버가 없습니다. 그런데 독자가 ▶ Run it live" 버튼을 누르면 **실제 LLM 앱이 뜨고, 페이지 안에서 바로 대화가 됩니다.** 뒤에서는 Flask 런처가 요청을 받아 k3s Deployment 를 0→1 로 스케일업하고, Traefik 이 라우팅을, cert-manager 가 TLS 를 붙인 뒤 iframe 으로 꽃힙니다. 유휴 상태가 되면 **파드는 자동으로 0 으로 회수되고,** 그래서 **서빙 비용은 0 으로 유지됩니다.**

GPU 가 아니라 CPU 환경이었다는 점은 감추지 않겠습니다. 하지만 제가 통과한 문제는 GPU 여부와 무관합니다 — **콜드스타트를 사용자가 견딜 수 있는 시간 안에 끝내는 것, 유휴 자원을 언제 회수할지 정하는 것, 스케일 0 상태에서 들어온 요청을 흘리지 않고 붙잡아 두는 것.** 온디맨드 서빙에서 실제로 부딪히는 건 이런 것들이었고, 저는 그것을 제 손으로 겪었습니다.

3. 고객 환경의 제약 위에서 돌아가게 만드는 일

채용공고는 "고객마다 다른 vpc 토폴로지, 보안 정책, 리전 제약 위에서 LLM 서빙 인프라를 프로덕션 레벨로 올려야 한다" 고 말합니다. 여기서 어려운 건 기술이 아니라 **내가 통제할 수 없는 환경입니다.**

제 인프라는 제가 다 아는 환경이라, 모르는 환경에서 깨지는 경험은 다른 데서 쌓았습니다.

Microsoft M365 글로벌 기술지원에서 20 명 중 전 지표 1 위를 했습니다(해결률 92.7%, 불만족 0%). 낮은 테넌트·보안 정책, 이미 화가 난 고객 앞에서 문제를 재현하고 원인을 좁혀야 하는 자리였습니다. 라이선스·인증·권한(OAuth·테넌트 관리)과 네트워크 이슈를 영어로 진단했고, 매번 **재현 절차와 해결 과정을 문서로 남겨 팀 자산으로 축적했습니다.**

그리고 AWS(EC2·Route 53) 위에 운영하던 서비스 전체를 워크로드·비용·아키텍처 관점에서 다시 계산해 OCI 로 무중단 단독 이관했습니다. 옮기는 순서를 정하는 일, 양쪽을 함께 살려두는 이중 운영, 네트워크·스토리지 의존성을 끊는 지점, "돌아는 가지만 예전 같지 않다"를 발견하는 관측 체계 — **전환은 기술 문제이기 전에 판단과 순서의 문제**라는 것을, 직접 옮겨보고 알았습니다.

4. 제품을 쓰다 만난 문제를, 재현 가능한 피드백으로 바꿨습니다

채용공고의 마지막 문장이 인상 깊었습니다 — "현장에서 반복되는 배포 패턴을 제품 기능으로 내재화하도록 제품팀에 직접 제안한다."

저는 이미 Upstage 제품으로 그 일을 해봤습니다. BookToss(서울 25 개 구립도서관 통합검색 에이전트)를 Solar API 로 재구축하며 겪은 것들을 감정이 아니라 재현 절차가 붙은 공개 피드백으로

정리했습니다(bookseal.github.io/Booktoss/feedback.html).

신용카드 없이 키가 발급되는 낮은 진입 장벽, base_url 한 줄로 갈아끼워지는 OpenAI 호환 설계의 강점, 그리고 응답이 근거인지 추측인지 구분하기 어렵다는 한계까지 버전 태그와 PR 로 남겨 누구나 재현할 수 있게 했습니다.

제품을 쓰다 막힌 지점을 재현 가능한 기록으로 바꾸는 일 — 그것이 이 포지션이 파트너 현장에서 하는 일이고, 같은 기록이 파트너가 Runbook 만 보고 독립 배포할 수 있는 기술 패키지가 됩니다.

5. 연차에 대해서는 감추지 않겠습니다

필수 사항의 "클라우드 3 년 이상"에 대해 정직하게 말씀드립니다. 제 개발 경력의 연차는 길지 않고, AWS 사용 폭도 EC2·Route 53 에 머물러 있습니다. 부풀리지 않겠습니다.

대신 그 시간에 무엇을 했는지로 말씀드립니다. 42 Seoul 에서 UNIX 시스템·네트워크·동시성을 **c 로 밑바닥부터 구현**했고(2 년), 그 뒤 클러스터를 세워 **2 년을 혼자 운영**했습니다 — 새벽에 서버가 죽으면 고칠 사람이 저뿐인 환경에서 알림→원인분석→복구→재발방지 루프를 위임 없이 반복했습니다. 그동안 Terraform 으로 인프라를 코드로 정의하고, GitOps 파이프라인을 표준화하고, LLM 앱을 온디맨드로 서빙하고, 글로벌 기술지원에서 전 지표 1 위를 했습니다.

말을 믿어달라고 부탁드립니다. 지금 열려 있습니다.

- status.bit-habit.com — 전 서비스의 실시간 헬스 대시보드. 지금 무엇이 살아있고 무엇이 죽었는지 그대로 보입니다.
- infra.bit-habit.com — 클러스터 구조·트래픽 경로·장애 지점을 다이어그램으로 설명한 아키텍처 문서.

깨져 있으면 깨진 채로 보이도록 공개해 두었습니다. **관측되지 않는 시스템은 운영이 아니라 방치라고 보기 때문입니다.**

연차로 증명할 수 없다면, 굴러가는 것으로 증명하겠습니다.

포트폴리오 — bit-habit.com